iPerf, TCP Statistics, and Congestion Control

Assignment 2

Marko Kovacevic

CS 42200: Computer Networks
Purdue University · Fall 2026
Instructor: Vamsi Addanki
TAs: Youngsuk Kim, Xiao Luo, and Albert Vo

October 21, 2026

Abstract

Two 15-second transfers were made for each of CUBIC, Reno, and BBR at ten public iperf3 destinations. The client measured goodput (acknowledged data per second) and sampled TCP statistics on the sending data socket. BBR had the lowest pooled median round-trip time (RTT) at seven destinations. The goodput leader changed between runs at five destinations. On a path with a minimum RTT of 186.4 ms, CUBIC's window was still growing at the end of both runs. That observation motivated a proposed window rule using acknowledged rate, RTT, and retransmissions.

1. Introduction

TCP congestion control limits how much unacknowledged data a sender may have in flight. A small window can leave a path underused, while a large one can build a queue or cause loss. The Python client sends to public iperf3 servers under CUBIC, Reno, and BBR and samples goodput, RTT, the congestion window, and retransmissions on its data socket. Interval measurements show whether a rate difference persists through a transfer or appears only while a sender is growing its window.

2. Methodology

2.1. Client and measurement setup

The server list came from iperf3serverlist.net. The selection loop randomized its entries and chose one port within each range. Site names follow the server list. The script could try up to 50 candidates. It reached all 10 requested complete destinations after 19 attempts, with 2 runs per algorithm at each.

Seven candidates refused the connection and two timed out. Another seven transfers had completed on these candidates before a later test failed. They were excluded because their destinations did not have all six runs. The client replaced failed candidates. Completed low-rate transfers remained in the comparison because the sender could not distinguish rate limiting from a slow path.

For each transfer, the client opened an iperf3 control connection, sent its 37-byte cookie, exchanged length-prefixed JSON parameters, and opened a data connection. It sent data for 15 seconds, then completed the result exchange and final state. It used neither an iperf3 binary nor a custom receiver. Before connecting the data socket, the client set `TCP_CONGESTION` and read back the selected algorithm. The data sender used the same destination, duration, and 0.5 second sampling interval for all three algorithms.

The six test orders were CUBIC–Reno–BBR, BBR–CUBIC–Reno, Reno–BBR–CUBIC, CUBIC–BBR–Reno, Reno–CUBIC–BBR, and BBR–Reno–CUBIC. The client rotated through them by candidate and repetition. At the Toronto representative, repetition 1 ran Reno, BBR, CUBIC, and repetition 2 ran CUBIC, BBR, Reno.

The host ran Linux kernel 6.17.0-35-generic with CUBIC as its default algorithm. CUBIC, Reno, and BBR were available for selection. These measurements were taken without a VPN. Docker fixes the Python environment on Ubuntu 24.04 but shares the host kernel. On this host, the `tcp_bbr` module had to be loaded before BBR could be selected.

2.2. TCP statistics and goodput

Every sample came from `getsockopt(TCP_INFO)` on the **data socket**. The decoder uses the host's Linux `struct tcp_info` layout. Its byte offsets are 68 for `tcpi_rtt`, 72 for `tcpi_rttvar`, 80 for `tcpi_snd_cwnd`, 100 for `tcpi_total_retrans`, 104 for `tcpi_pacing_rate`, and 120 for `tcpi_bytes_acked`. RTT and RTT variation are stored in microseconds and converted to milliseconds for plots. `snd_cwnd` is measured in TCP segments, so multiplying it by `snd_mss` gives its byte limit. `tcpi_total_retrans` is cumulative. Its increase between samples is the loss proxy, a count of retransmissions rather than confirmed lost packets.

Interval goodput is `8 × (current bytes_acked - previous bytes_acked) / actual elapsed seconds`. The monotonic clock supplies that elapsed time. Goodput counts acknowledged application data, whereas `send()` counts bytes passed to the socket. Each run records UTC time, elapsed time, units, destination, algorithm, run identifier, counters, and derived values.

The summary table pools measured intervals from the 2 repetitions for each destination and algorithm. A final shorter interval is included if it lasts at least one quarter of the chosen interval. The 95th percentile uses linear interpolation between adjacent ordered samples.

2.3. AI acknowledgment

I used Codex to work through the iperf3 protocol, debug the client, and edit this report. I checked the `tcp_info` offsets against the host kernel and tested a complete transfer against a public server.

3. Question 1: Throughput application

All ten selected servers completed two runs under each algorithm. The figures show acknowledged goodput from the data socket over time. Solid lines show the first run and dashed lines the second. Color identifies the algorithm. The table gives the minimum, median, mean, and 95th-percentile interval goodput for each destination and algorithm.

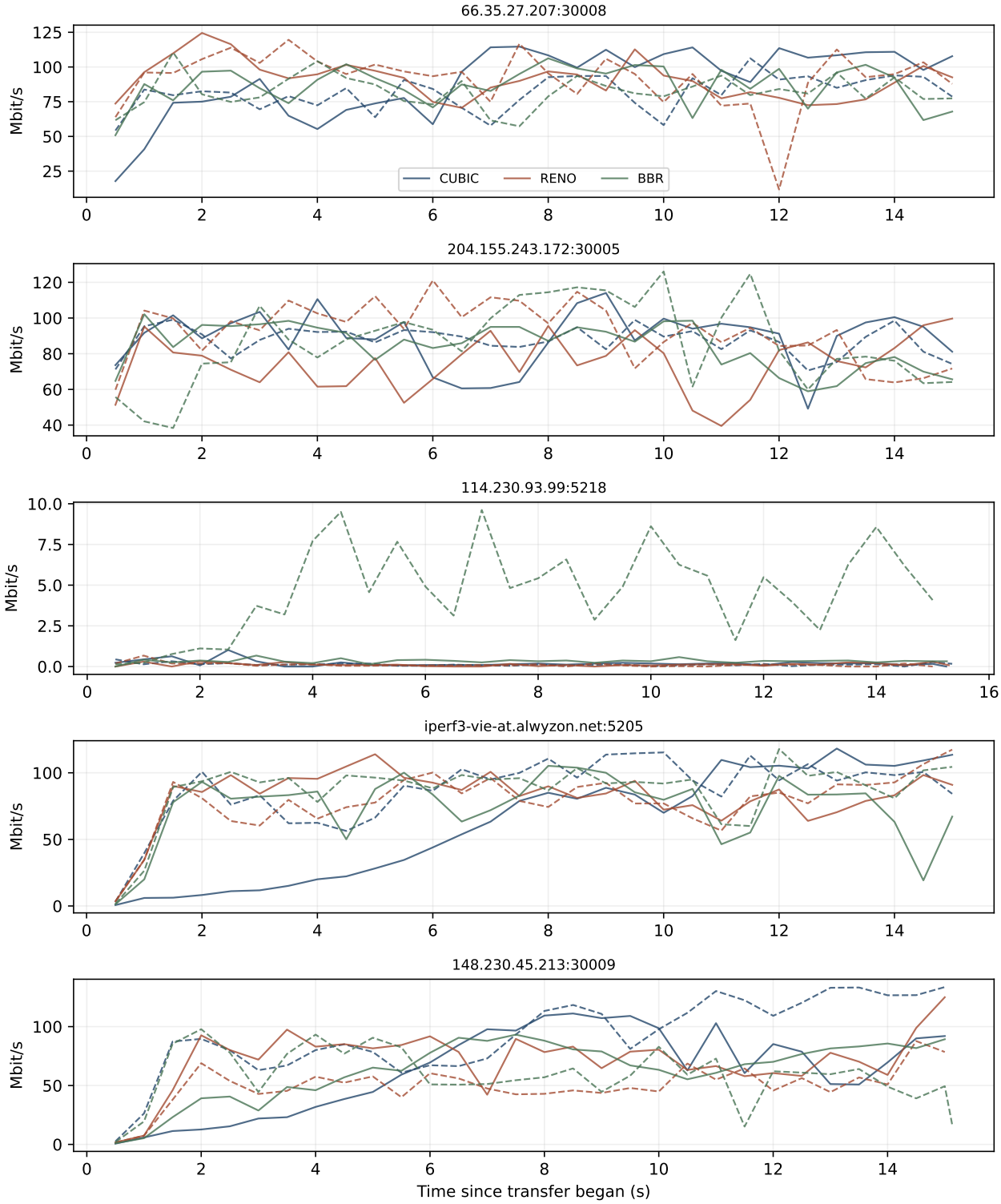


Figure 1: Acknowledged goodput over time for destinations 1–5.

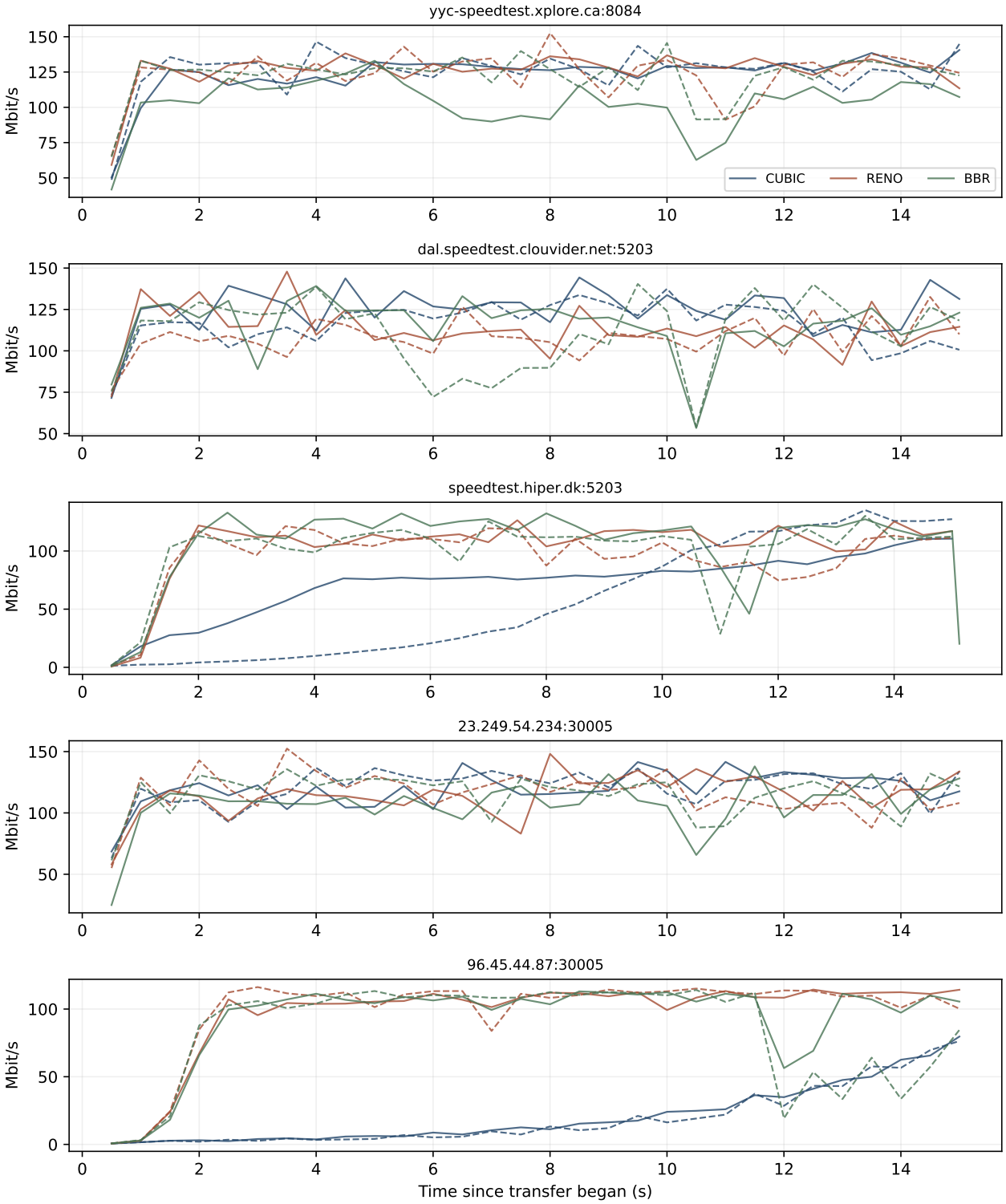


Figure 2: Acknowledged goodput over time for destinations 6–10.

Acknowledged goodput (Mbit/s)

Destination	Port	TCP	Min	Median	Mean	P95
66.35.27.207	30008	cubic	17.8	87.1	85.6	113.6
66.35.27.207	30008	reno	11.8	94.7	91.5	116.4
66.35.27.207	30008	bbr	50.8	85.3	84.7	102.1
204.155.243.172	30005	cubic	49.2	89.8	88.1	103.7
204.155.243.172	30005	reno	39.5	83.8	83.7	111.7
204.155.243.172	30005	bbr	38.4	87.0	85.4	115.6
114.230.93.99	5218	cubic	0.0	0.1	0.2	0.4
114.230.93.99	5218	reno	0.0	0.1	0.1	0.3
114.230.93.99	5218	bbr	0.0	0.5	2.5	8.6
iperf3-vie-at.alwyzon.net	5205	cubic	0.7	84.6	74.9	113.7
iperf3-vie-at.alwyzon.net	5205	reno	3.4	84.4	80.7	104.9
iperf3-vie-at.alwyzon.net	5205	bbr	1.3	87.9	80.8	103.9
148.230.45.213	30009	cubic	0.7	82.7	78.0	130.4
148.230.45.213	30009	reno	1.6	58.6	61.0	92.9
148.230.45.213	30009	bbr	0.9	62.5	60.8	90.6
yyc-speedtest.xplore.ca	8084	cubic	48.6	127.9	124.5	140.7
yyc-speedtest.xplore.ca	8084	reno	59.2	128.2	125.0	137.8
yyc-speedtest.xplore.ca	8084	bbr	41.7	117.0	113.0	133.2
dal.speedtest.clouvider.net	5203	cubic	71.6	123.0	120.6	139.5
dal.speedtest.clouvider.net	5203	reno	72.6	109.6	110.5	132.7
dal.speedtest.clouvider.net	5203	bbr	53.4	119.2	113.4	138.8
speedtest.hiper.dk	5203	cubic	1.4	76.2	64.9	125.7
speedtest.hiper.dk	5203	reno	0.7	109.3	100.1	121.7
speedtest.hiper.dk	5203	bbr	1.0	112.4	103.4	130.2
23.249.54.234	30005	cubic	63.1	122.2	120.2	136.9
23.249.54.234	30005	reno	55.1	117.8	115.4	136.2
23.249.54.234	30005	bbr	24.9	114.6	111.4	131.7
96.45.44.87	30005	cubic	0.7	10.8	20.4	65.8
96.45.44.87	30005	reno	0.7	109.7	97.9	114.3
96.45.44.87	30005	bbr	0.6	106.1	88.6	112.6

Each row pools the half-second samples from two completed runs.

At `114.230.93.99`, all three algorithms had pooled median goodput below 1 Mbit/s. At `yyc-speedtest.xplore.ca`, all three exceeded 100 Mbit/s.

4. Question 2: TCP statistics during transfer

The detailed plots use 204.155.243.172:30005. The selection rule chooses the first complete destination with a CUBIC retransmission, giving the loss scatter nonzero values. The three-algorithm comparison uses that same destination.

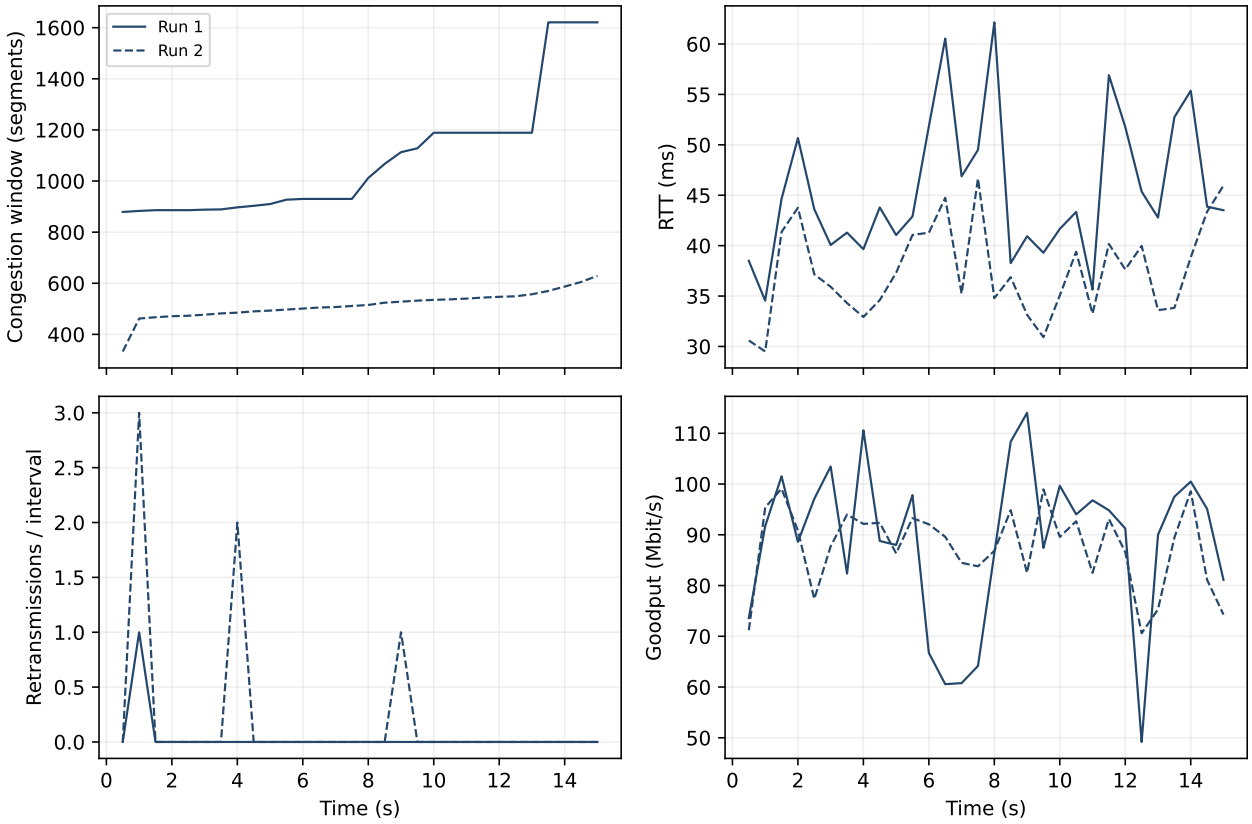


Figure 3: CUBIC data-socket congestion window, RTT, retransmissions per sample, and goodput over time.

`snd_cwnd` limits how many segments may be unacknowledged. If the window is too small to fill the path, it caps goodput. Once the path is full, a larger window can leave more data waiting in a queue and raise RTT. The measured RTT also includes propagation delay, which the sender cannot separate from queueing. An increase in `tcpi_total_retrans` means segments were resent, possibly after loss or reordering. The traces cover slow start, when the window grows rapidly, and recovery after retransmission as well as congestion avoidance.

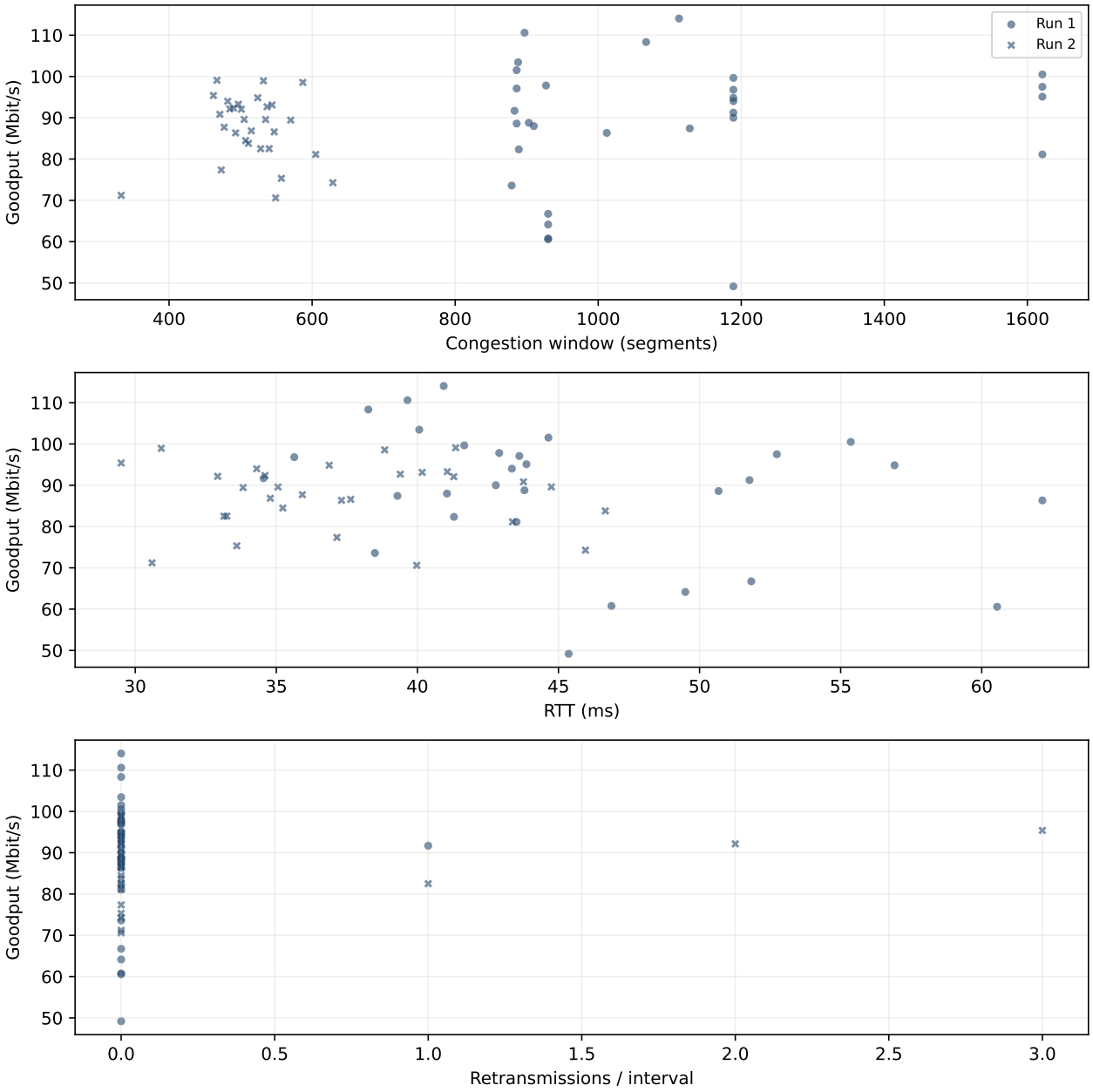


Figure 4: CUBIC congestion window, RTT, and retransmission signal versus acknowledged goodput.

At the Toronto-listed server, `d02-r1-cubic` had one retransmission and a median window of 971 segments. `d02-r2-cubic` had six retransmissions and a median window of 513 segments. Their median goodputs were close, 91.5 and 89.5 Mbit/s. The three retransmission intervals in the second run had goodputs of 95.4, 92.2, and 82.5 Mbit/s, with no uniform immediate drop. The second run's median RTT was lower, 37.0 rather than 43.6 ms. A smaller window could have left less data queued, but `snd_cwnd` is only a limit, and the sequential traces do not show whether the path or competing traffic changed.

5. Question 3: Algorithm comparison and design

5.1. CUBIC, Reno, and BBR

Each metric uses a common scale across algorithm panels. Solid and dashed traces show repetitions.

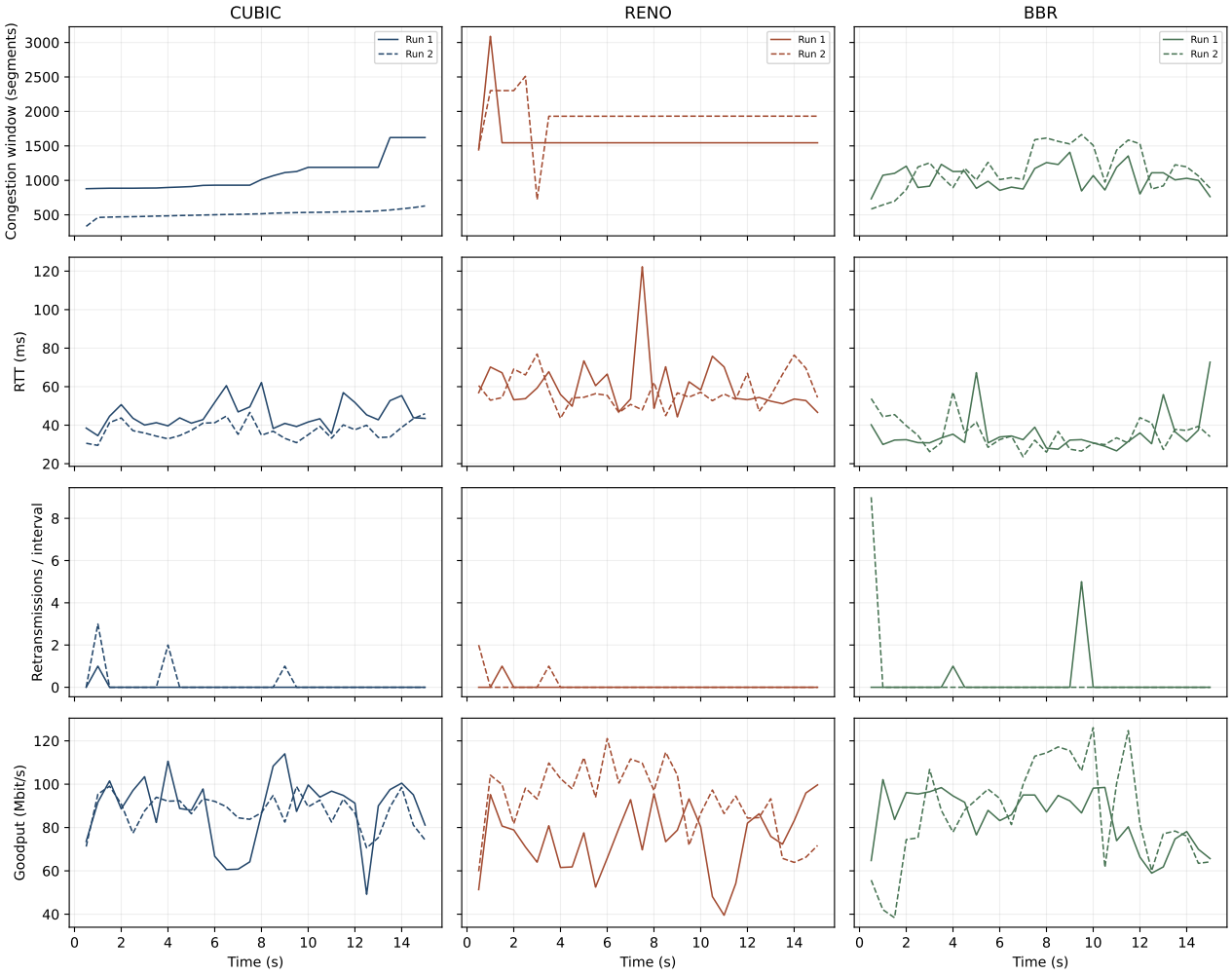


Figure 5: Time series for all three algorithms on one destination, with common scales by metric.

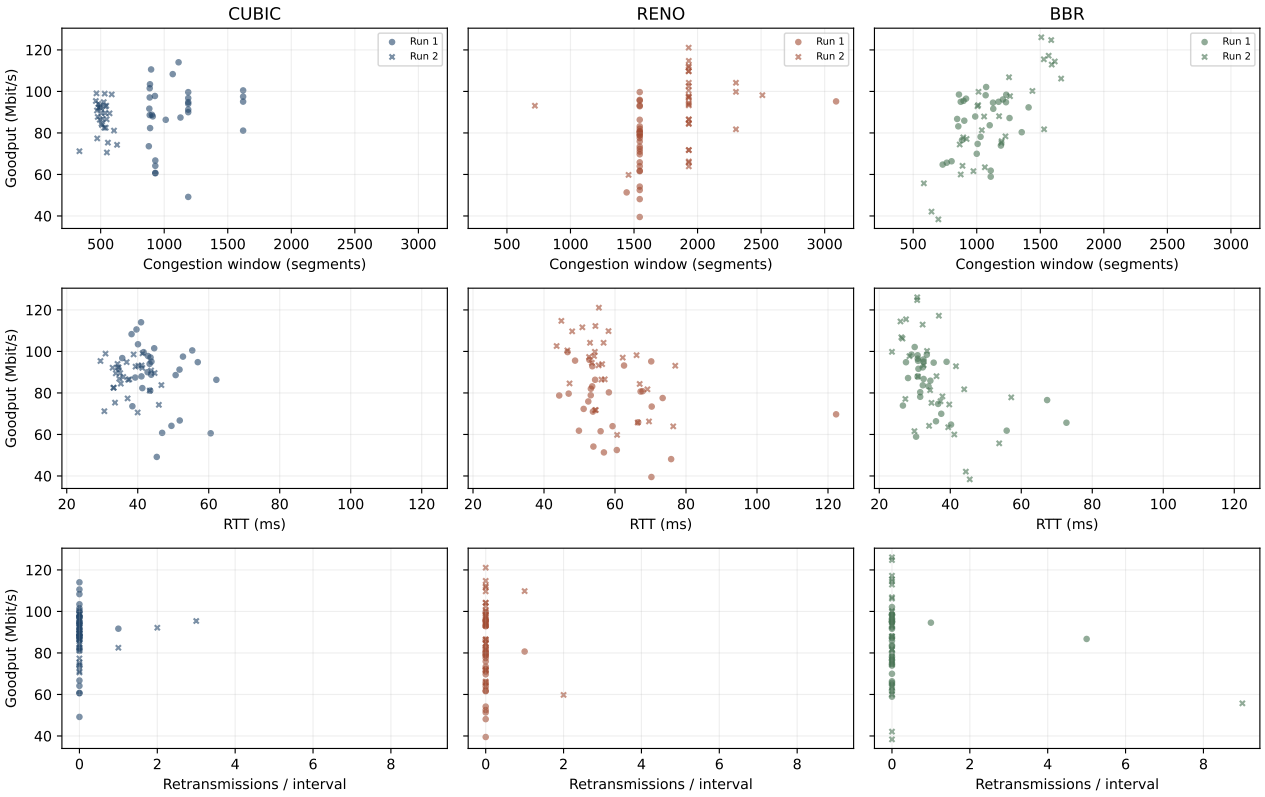


Figure 6: Congestion window, RTT, and retransmission signal versus goodput for all three algorithms.

Reno grows roughly one segment per RTT in congestion avoidance and reduces its window after loss (RFC 5681). CUBIC uses a cubic window-growth function for paths with a large bandwidth-delay product (BDP) (RFC 9438). At a given delivery rate, BDP is rate multiplied by RTT. It estimates how much data must be in flight to sustain that rate. BBR estimates delivery bandwidth and minimum RTT and uses pacing to spread transmissions over time (Cardwell et al.).

Median RTT (ms)

Destination	CUBIC	Reno	BBR
66.35.27.207	52.8	59.5	43.2
204.155.243.172	41.0	55.4	32.5
114.230.93.99	220.1	241.9	216.3
iperf3-vie-at.alwyzon.net	126.1	150.2	123.6
148.230.45.213	126.8	159.1	129.3
yyc-speedtest.xplore.ca	61.9	79.5	52.9
dal.speedtest.clouvider.net	49.5	55.1	37.5
speedtest.hiper.dk	127.2	150.0	135.3
23.249.54.234	59.6	63.4	51.0
96.45.44.87	191.2	198.2	199.2

Each value pools the half-second samples from two completed runs.

Across the ten destinations, CUBIC had the highest pooled median goodput at four, Reno at three, and BBR at three. BBR had the lowest pooled median RTT at seven. The leader changed between runs at five destinations, which the pooled counts hide. Taking a median within each run before comparing destinations changes the goodput tally to five for CUBIC, two for Reno, and three for BBR.

At the Toronto-listed destination, BBR had 32.5 ms pooled median RTT and 87.0 Mbit/s goodput. CUBIC had 41.0 ms and 89.8 Mbit/s. BBR's RTT was lower in both repetitions, while the goodputs were similar. Reno's run median goodput changed from 78.2 to 95.8 Mbit/s, crossing CUBIC's 91.5 and 89.5 Mbit/s. BBR's lower RTT is consistent with less queueing, though the sequential runs cannot isolate queueing from changing path conditions.

The goodput ordering at `speedtest.hiper.dk` held in both repetitions. CUBIC's run medians were 77.4 and 40.1 Mbit/s, Reno's were 112.1 and 106.5, and BBR's were 118.7 and 110.4. Reno and BBR had pooled median windows of 4,003 and 3,618 segments, compared with CUBIC's 864. Their pooled median goodputs exceeded 109 Mbit/s while CUBIC's was 76.2 Mbit/s. BBR's pooled RTT was lower than Reno's, 135.3 versus 150.0 ms, but higher than CUBIC's 127.2 ms. The larger window limits for Reno and BBR coincided with higher goodput here.

At `96.45.44.87`, the smallest observed RTT was 186.4 ms. Reno's 109.7 Mbit/s pooled median goodput multiplied by that RTT gives an approximate operating-point bandwidth-delay product of 2.56 MB, or 1,765 segments at the measured 1,448-byte maximum segment size (MSS). By contrast, CUBIC's median window was 210 segments and its goodput was 10.8 Mbit/s. At its 191.2 ms median RTT, filling that window would permit about 13 Mbit/s, a rough estimate because the medians come from different samples. By the end of each CUBIC run, `snd_cwnd` reached about 1,410 segments and interval goodput reached 76–80 Mbit/s. Reno and BBR had median windows of 3,974 and 5,481 segments and goodputs of 109.7 and 106.1 Mbit/s. Their run medians were above 100 Mbit/s in both repetitions, whereas CUBIC's were 11.9 and 10.1. The rising CUBIC window supports a slow-ramp explanation for this 15-second result.

All three algorithms were slow at `114.230.93.99`. CUBIC and Reno each had about 0.1 Mbit/s pooled median goodput, with 61 and 69 retransmissions across two runs. BBR had 0.5 Mbit/s and 1,523 retransmissions. Its two run medians were 0.3 and 4.9 Mbit/s. The BBR run medians differ by more than an order of magnitude, making its pooled value a poor basis for ranking.

5.2. Proposed algorithm

At `96.45.44.87`, CUBIC's window was still growing when both 15-second runs ended. Starting at two segments and adding one segment per 186 ms RTT would take about 5.5 minutes to reach the estimated 1,765-segment operating point. I therefore propose a congestion-control rule that begins with slow start, pauses when RTT rises, and halves the window after a retransmission. It leaves slow start after two RTTs without meaningful growth in acknowledged rate.

The variable `cwnd` is measured in MSS segments, starts at two, and has a lower bound of two. `minimum_RTT` is the lowest sampled RTT. The RTT and acknowledged-rate estimates each use one quarter of the new sample and three quarters of the previous estimate. A sampled ACK rate is newly acknowledged bytes divided by elapsed time. `flat_rounds` counts consecutive RTTs with less than 5% growth in smoothed rate. `delay_cut_used` prevents another cut for high delay until delay falls. Loss backoff also sets `delay_cut_used`. A high-delay reading means smoothed RTT exceeds `minimum_RTT` by more than the larger of 2 ms and 20% of `minimum_RTT`.

```
cwnd = 2 MSS segments
mode = startup
flat_rounds = 0
delay_cut_used = false
minimum_RTT = unset
smoothed_RTT = unset
smoothed_rate = unset
once per RTT:
    if retransmissions increased:
        mode = avoidance
        cwnd = max(2, floor(0.50 × cwnd))
        flat_rounds = 0
        delay_cut_used = true
        continue
    if RTT invalid or sampled_ACK_rate <= 0:
        flat_rounds = 0
        hold cwnd
        continue
    if minimum_RTT is unset:
        minimum_RTT = sampled_RTT
        smoothed_RTT = sampled_RTT
        smoothed_rate = sampled_ACK_rate
        hold cwnd
        continue
    minimum_RTT = min(minimum_RTT, sampled_RTT)
    smoothed_RTT = 0.75 × old_smoothed_RTT + 0.25 × sampled_RTT
    previous_rate = smoothed_rate
    smoothed_rate = 0.75 × old_smoothed_rate + 0.25 × sampled_ACK_rate
    queue_delay = smoothed_RTT - minimum_RTT
    queue_limit = max(2 ms, 0.20 × minimum_RTT)
    if previous_rate > 0 and smoothed_rate < 1.05 × previous_rate:
        flat_rounds = min(2, flat_rounds + 1)
    else:
        flat_rounds = 0
    if mode == startup and flat_rounds == 2:
        mode = avoidance
        if queue_delay > queue_limit:
            cwnd = max(2, floor(0.90 × cwnd))
            delay_cut_used = true
        else:
            hold cwnd
    else if mode == startup and queue_delay > queue_limit:
        hold cwnd
    else if mode == startup:
        cwnd = 2 × cwnd
    else if queue_delay > queue_limit:
        if flat_rounds == 2 and not delay_cut_used:
            cwnd = max(2, floor(0.90 × cwnd))
            delay_cut_used = true
        else:
            hold cwnd
    else:
        delay_cut_used = false
        cwnd = cwnd + 1 segment
```

Startup grows quickly to avoid the long ramp seen on the 186 ms path. High delay alone pauses growth. Two flat-rate RTTs end startup even if delay stays low. A retransmission halves `cwnd`. In avoidance, growth is one segment per RTT while delay stays low. High delay with a flat rate permits one 10% cut, smaller than the loss response because RTT inflation is an ambiguous signal. The smoothing weight, two-round check, and percentage thresholds are design choices, not values fitted to these measurements.

Consider two saturated flows following this rule at a stable shared bottleneck. With similar RTTs, both add one segment per RTT while delay is low. A flow that sees a flat rate and high delay can make one 10% cut before holding until delay falls. A retransmission halves only the affected flow's window. Repeated growth and backoff can make their rates oscillate instead of settling at one value. A shorter-RTT flow gets more growth steps per second and can take a larger share. A loss-based competitor can keep the queue above this rule's delay threshold and leave it on hold.

The rule assumes a single path and an application that keeps sending. Missing RTT or ACK-rate samples hold `cwnd`, with retransmissions still taking priority. Delayed feedback can let startup overshoot on a long-RTT path, and a route change can make the fixed minimum RTT stale. An implementation needs a window-limited check to avoid growing `cwnd` when the receiver is the limit.

6. Limitations and reproducibility

Public iperf3 servers can be busy or rate limited, so the observed rate reflects both server and path conditions. The paths and competing traffic were not controlled, and matched tests ran sequentially. The 15-second duration emphasizes startup, so longer transfers could rank the algorithms differently. `TCP_INFO` fields and congestion algorithms depend on the Linux host kernel even when Python runs in Docker.

The code is at github.com/kovaceviccz/cs422-assignment-2. The main script runs the measurements and calls the plotting functions. Reproduction requires Linux with CUBIC, Reno, and BBR available to TCP sockets, Python 3.12, the packages in `requirements.txt`, and Typst 0.14.2. The Dockerfile begins with `FROM ubuntu:24.04` and uses the host kernel. On the measured host, BBR required `sudo modprobe tcp_bbr`. With those prerequisites and an Internet connection, run:

```
git clone https://github.com/kovaceviccz/cs422-assignment-2
cd cs422-assignment-2
python3 -m pip install -r requirements.txt
python3 assignment2.py listed_iperf3_servers.json \
  --destinations 10 --duration 15 --interval 0.5 --repetitions 2
```

7. Conclusion

The client completed two runs per algorithm at each of ten public iperf3 destinations and recorded acknowledged goodput with live TCP statistics from the data socket. BBR had the lowest pooled median RTT at seven destinations, while the goodput leader changed between runs at five. On the 186.4 ms path, CUBIC's window was still growing when both 15-second runs ended, whereas Reno and BBR each had run median goodputs above 100 Mbit/s. To address the slow ramp seen there, the proposed rule begins with slow start and uses RTT, acknowledged rate, and retransmissions to decide when to pause or reduce its window.